

Verteilte Systeme, Containerisierung

Vorwort:

Grundlage für diese Aufgabe ist die Präsentation «03_Deployments_Services_ConfigMaps»

Dauer: 80 Minuten

Besprechung: 20 Minuten

Total: 100 Minuten

Block 3 setzt den Kubernetes-Einstieg aus Block 2 voraus. Statt nochmals eine Standard-Applikation aufzubauen, wird nun der erste Dispatch-City-Stand aus bereitgestelltem Code gebaut, deklarativ deployt und als zusammenhängendes System geprüft.

Arbeitsmaterial: öffentliches GitHub-Template «vsc-dispatch-city-03-foundation», verbindlicher Stand «v1.0.0»: <https://github.com/SwitzerChees/vsc-dispatch-city-03-foundation>. Pro Einzelperson oder Zweierteam wird daraus ein eigenes, fortlaufendes Projekt-Repository erstellt.

Setup-Hinweis für Windows und macOS: Verwenden Sie den Cluster «teko-k8s» aus Block 2. Auf Windows ist WSL2 mit Docker-Desktop-Integration am stabilsten; PowerShell funktioniert mit den einzeln aufgeführten Befehlen. Makefile und .sh-Skripte benötigen Git Bash, WSL oder ein installiertes make.

```
k3d cluster list
# vorhandenen, gestoppten Cluster starten:
k3d cluster start teko-k8s
kubectl config use-context k3d-teko-k8s
kubectl get nodes -o wide

# nur falls der Cluster fehlt:
k3d cluster create teko-k8s --agents 2 --wait
```

Falls Port 3000 belegt ist, verwenden Sie z. B. «3002:3000». Die Control API bleibt in Block 3 bei einer Replik, weil jede Replik noch einen eigenen In-Memory-Zustand hätte.

Offizielle Dokumentation

- <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- <https://kubernetes.io/docs/concepts/services-networking/service/>
- <https://kubernetes.io/docs/concepts/configuration/configmap/>
- <https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>

Aufgabe 1)

Foundation übernehmen und Systemgrenze verstehen

Übernehmen Sie den Starter und prüfen Sie nur die Voraussetzungen aus Block 2. Der Cluster wird wiederverwendet; ein neues NGINX-Lab ist nicht vorgesehen.

- Erstellen Sie aus dem GitHub-Template ein eigenes Repository und verwenden Sie den Release «v1.0.0».
- Prüfen Sie Kontext und Nodes des Clusters «teko-k8s». Erstellen Sie ihn nur neu, falls er nicht mehr vorhanden ist.
- Erklären Sie die Rollen von «dashboard» und «control-api» sowie den Grund für genau eine Control-API-Replik.

Antwort / Notizen:

"dashboard" ist das Nuxt-Frontend, das die Simulation visualisiert und per HTTP/SSE mit der Control API kommuniziert.

"control-api" haelt den gesamten Simulationszustand (Bestellungen, Restaurants, Kuriere) im Arbeitsspeicher (In-Memory) und stellt ihn ueber REST/SSE bereit.

Genau eine Replik ist noetig, weil der Zustand nicht zwischen mehreren Instanzen geteilt wird (kein gemeinsamer Speicher, keine Datenbank in diesem Block) - zwei Repliken haetten zwei unabhaengige, widerspruechliche Zustaende.

Aufgabe 2)

Images bauen und Kustomize-Ausgabe lesen

Bauen Sie aus dem bereitgestellten Code zwei Images, laden Sie diese in den vorhandenen k3d-Cluster und rendern Sie den geplanten Kubernetes-Stand vor dem Deployment.

- Ordnen Sie jedem Image das zugehörige Dockerfile und den Tag «local» zu.
- Importieren Sie beide Images in «teko-k8s» und erklären Sie die Rolle von «imagePullPolicy: IfNotPresent».
- Führen Sie «kubectrl kustomize deploy/overlays/block-03-standalone» aus. Welche Ressourcen entstehen durch Base und Overlay?

Antwort / Notizen:

control-api <- build/go-service.Dockerfile (--build-arg SERVICE=control-api), Tag "local".

dashboard <- apps/dashboard/Dockerfile, Tag "local".

imagePullPolicy: IfNotPresent: Kubernetes laedt ein Image nur dann von einer Registry, wenn es nicht schon lokal auf dem Node vorhanden ist. Fuer lokal gebaute Images (kein Push in eine Registry) ist das noetig, sonst wuerde kubelet ImagePullBackOff werfen.

kubectrl kustomize rendert die fertigen YAML-Manifeste, ohne sie anzuwenden. Base enthaelt die Grundressourcen (Deployments, Services, ConfigMap) fuer beide Apps; das Overlay block-03-standalone passt sie an (Namespace, Labels, Image-Tags) und kann Ressourcen ergaenzen/patchen.

```
# macOS/Linux/Git Bash/WSL:
make images
CLUSTER=teko-k8s make load
```

```
kubect1 kustomize deploy/overlays/block-03-standalone

# PowerShell oder ohne make:
docker build -t food-delivery-control-api:local --build-arg SERVICE=control-api -f ./build/go-
service.Dockerfile .
docker build -t food-delivery-dashboard:local ./apps/dashboard
k3d image import -c teko-k8s food-delivery-control-api:local food-delivery-dashboard:local
kubect1 kustomize deploy/overlays/block-03-standalone
```

Aufgabe 3)

Manifeste als Vertrag deployen

Lesen Sie die Referenzen zwischen den Ressourcen und wenden Sie danach den gerenderten Projektstand an.

- Verfolgen Sie für beide Anwendungen: Service-Selector → Pod-Labels sowie «targetPort» → benannter Container-Port.
- Verfolgen Sie bei der Control API: «envFrom» → ConfigMap «simulation-config» und die konfigurierten Probes.
- Deployen Sie das Overlay und belegen Sie mit «rollout status» und «get ... -o wide», dass beide Workloads bereit sind.

Antwort / Notizen:

Service-Selector (z.B. app: control-api) matcht die Labels auf den Pods aus dem Deployment-Template - nur Pods mit exakt diesen Labels werden als Endpoints aufgenommen. targetPort verweist auf einen benannten Container-Port statt einer festen Zahl, das entkoppelt Service und Deployment.

envFrom mit configMapRef laedt alle Key-Value-Paare aus der ConfigMap "simulation-config" als Umgebungsvariablen (z.B. APP_MODE, TICK_MS). Readiness-Fehlschlag entfernt den Pod aus den Service-Endpoints; Liveness-Fehlschlag killt und startet den Container neu.

Nachweis: kubectl apply -k ... , rollout status zeigt "successfully rolled out" fuer beide Deployments, kubectl get ... -o wide zeigt READY 1/1 fuer beide.

```
kubectl apply -k deploy/overlays/block-03-standalone
kubectl -n food-delivery get deploy,pods,svc,cm -o wide
kubectl -n food-delivery rollout status deployment/control-api
kubectl -n food-delivery rollout status deployment/dashboard
kubectl -n food-delivery describe deployment/control-api
```

Aufgabe 4)

Service, EndpointSlice und DNS als Kette prüfen

Prüfen Sie nicht nur, ob ein Service existiert, sondern ob er tatsächlich einen bereiten Pod erreicht und im Cluster per DNS auflösbar ist.

- Vergleichen Sie Service-Selector, Pod-Labels und die EndpointSlice des Services «control-api».
- Rufen Sie «/health/ready» aus einem temporären Curl-Pod über Kurzname und vollständigen DNS-Namen auf.
- Erklären Sie, warum der DNS-Name auf dem Host nicht funktioniert und welche Wirkung eine fehlgeschlagene Readiness Probe hätte.

Antwort / Notizen:

Service-Selector und Pod-Labels muessen exakt uebereinstimmen (z.B. app: control-api), sonst bleibt die EndpointSlice leer, obwohl der Service existiert - ein sehr haeufiger Fehler ("Service exists, but 0 endpoints").

Der Kurzname (control-api) funktioniert nur innerhalb desselben Namespace, weil Cluster-DNS ihn relativ zur Search-Domain des anfragenden Pods auflöst; der vollstaendige Name (control-api.food-delivery.svc.cluster.local) funktioniert clusterweit.

Vom Host aus funktioniert weder Kurz- noch Vollname, weil CoreDNS nur innerhalb des Cluster-

Netzwerks erreichbar ist. Eine fehlschlagende Readiness Probe wurde den Pod aus der EndpointSlice entfernen - Running, aber ohne Traffic.

```
kubectl -n food-delivery get svc,endpointslice
kubectl -n food-delivery get pods --show-labels
kubectl -n food-delivery get endpointslice -l kubernetes.io/service-name=control-api

kubectl run dns-test --image=curlimages/curl:8.8.0 -n food-delivery --restart=Never --rm -i -- curl
-fsS http://control-api:8080/health/ready

# Vollständiger DNS-Name:
http://control-api.food-delivery.svc.cluster.local:8080/health/ready
```

Aufgabe 5)

ConfigMap, Probes und Rollout nachvollziehen

Ändern Sie das Laufzeitverhalten, ohne ein neues Image zu bauen, und beobachten Sie den Unterschied zwischen ConfigMap-Änderung und Pod-Rollout.

- Erklären Sie «APP_MODE», «TICK_MS», Readiness und Liveness anhand der vorhandenen Manifeste.
- Ändern Sie «TICK_MS» von «500» auf einen gut erkennbaren Wert und wenden Sie die Kustomize-Ausgabe erneut an.
- Prüfen Sie zunächst den bestehenden Pod. Lösen Sie danach bewusst «rollout restart» aus und weisen Sie den neuen Wert in den Startlogs nach.

Antwort / Notizen:

APP_MODE steuert den Betriebsmodus (standalone vs. distributed). TICK_MS bestimmt das Intervall der Simulationsschritte (z.B. 500 = alle 500ms ein Tick). Readiness zeigt, ob der Pod Traffic annehmen kann; Liveness zeigt, ob der Prozess noch lebt.

Nur die ConfigMap zu ändern und erneut zu applyn aktualisiert den Inhalt im Cluster, ändert aber NICHT die bereits laufenden Pods - deren Env-Variablen wurden beim Start einmalig gelesen und bleiben eingefroren.

```
kubectl diff -k deploy/overlays/block-03-standalone
kubectl apply -k deploy/overlays/block-03-standalone
kubectl -n food-delivery rollout restart deployment/control-api
kubectl -n food-delivery rollout status deployment/control-api
kubectl -n food-delivery logs deployment/control-api
```

Aufgabe 6)

Projektstand abnehmen und Architekturgrenze dokumentieren

Nehmen Sie den ersten Dispatch-City-Meilenstein sichtbar und technisch nachvollziehbar ab.

- Öffnen Sie Dashboard und Control API per Port-Forward, starten Sie die Simulation und beschreiben Sie den sichtbaren Ablauf.
- Dokumentieren Sie je eine Evidenz für Image/Deployment, Service/DNS sowie ConfigMap/Probes/Rollout.
- Begründen Sie, weshalb die In-Memory-Control-API bei einer Replik bleibt und welche späteren Komponenten diese Grenze auflösen.

Antwort / Notizen:

Ueber zwei Port-Forwards (Dashboard 3000, Control API 8081->8080) sind beide Komponenten lokal erreichbar; im Dashboard sieht man die laufende Simulation in Echtzeit ueber die SSE-Verbindung zur Control API.

Evidenz: (1) Image/Deployment - `kubectl get deploy,pods -o wide` zeigt READY 1/1 mit Image-Tag "local"; (2) Service/DNS - `Curl gegen http://control-api:8080/health/ready` liefert `{"status":"ok"}`; (3) ConfigMap/Probes/Rollout - geaenderter TICK_MS-Wert nach rollout restart in den Logs sichtbar.

Die In-Memory-Control-API bleibt bei einer Replik, weil der Zustand nicht persistiert/geteilt wird. Spaetere Bloecke loesen das auf: RabbitMQ entkoppelt Komponenten ueber Events, CloudNativePG/PostgreSQL macht den Zustand persistent und fuer mehrere Repliken nutzbar.

```
kubectl -n food-delivery get deploy,pods,svc,endpointslice,cm

# Terminal 1:
kubectl -n food-delivery port-forward service/dashboard 3000:3000

# Terminal 2:
kubectl -n food-delivery port-forward service/control-api 8081:8080

# Browser:
http://127.0.0.1:3000
http://127.0.0.1:8081/health/ready
```

Aufräumen

Löschen Sie nur den Namespace «food-delivery». Der Cluster «teko-k8s» wird für Block 4 weiterverwendet und nur bei einem vollständigen Reset entfernt.

```
kubectl delete namespace food-delivery
# Der Cluster bleibt für Block 4 bestehen.
# Nur bei vollständigem Reset:
k3d cluster delete teko-k8s
```